Phase 0 Final Report: Technical Contributions

Submitted by: Ms. Sibgha Mursaleen (VNIAS Technical Member - Backend)

Focus Areas: Exception Handling, Security, and Database/Cache Integration

1. Executive Summary of Achievements

During this three-week preparation phase, I successfully led and implemented the foundational security, environment validation, and database abstraction patterns for the VNIAS-IJILS platform. All core objectives have been coded, integrated, and verified against local simulated runtime environments.

2. Completed Milestones & Implementations

Module A: Global Exception Handling Framework

- **Custom Error Classes:** Defined domain-specific custom exceptions including ManuscriptNotFoundError, DuplicateEmailError, UnauthorizedRoleError, and StorageUploadError.
- **Global Interceptors:** Registered centralized exception handlers on the FastAPI application factory to capture unhandled panics and format them into clean, predictable JSON responses.
- **Input Validation Interceptor:** Overrode the standard RequestValidationError to return neat, field-by-field diagnostic lists with a 422 Unprocessable Entity status code.

Module B: Security, Secrets, and CORS Core

- **Fast-Failing Configuration System:** Configured a type-safe Settings schema using pydantic-settings to load secrets from .env. The application safely halts boot-up immediately if critical keys like DB_URL are missing.
- **Dynamic CORS Controls:** Implemented institutional origins management via CORSMiddleware using dynamic environment configurations to protect against unauthorized browser-side requests.
- **Native Cryptographic Engine:** Resolved a compatibility gap between modern bcrypt libraries and legacy passlib wrappers on Python 3.13. Re-engineered the cryptographic utilities (hash_password, verify_password) to run directly on high-performance native binary bindings.

Module C: Database and Caching Integration

- **Asynchronous Connection Pooling:** Built a non-blocking database session dependency injection system (get_db) utilizing SQLAlchemy's AsyncSession.
- **The Repository Pattern:** Isolated all database query actions from routing logic by constructing a modular ManuscriptRepository containing non-blocking CRUD operations.

- **Asynchronous In-Memory Caching:** Built an abstract caching client leveraging asynchronous Redis protocols (aioredis) with a 5-minute Time-To-Live (TTL) strategy to dramatically cut down database lookups for static assets.
- **Query Optimization:** Implemented thin-model query utilities (get_slim_author_manuscripts) that selectively fetch explicit attributes to protect system bandwidth and prevent memory bloat under load.

3. Review of Student Deliverables (HTML Email Templates)

As part of my Week 3 quality assurance duties, I conducted a thorough technical code review of the 15 HTML email templates designed by the Science Academy student track:

- **General Assessment:** The structural layouts successfully utilize robust table-grid frameworks (<table>) which are critical for compatibility with legacy email clients (Outlook, Gmail).
- **Identified Corrections (Passed to Students):**
    1. **Token Standardization:** Instructed students to change variable brackets (e.g., {{Name}} to snake_case {{recipient_name}}) to cleanly align with backend database keys.
    2. **CAN-SPAM Footer Compliance:** Highlighted a missing physical address and dynamic unsubscribe link placeholder ({{unsubscribe_link}}) in the footers.
    3. **Plain-Text Fallbacks:** Prompted the addition of plain-text fallback comment summaries at the top of each .html file to avoid email client spam filters.

4. Technical Stack & Readiness Verification

The backend team's local development environment has been successfully configured and verified with:

- **FastAPI + Uvicorn ASGI Server** (running cleanly on Port 8080).
- **SQLAlchemy 2.0 Async Engine** with aiomysql drivers.
- **In-Memory Caching Fallback Engine**.

Verdict: The core architecture is completely optimized, secured, and ready for Phase 1 production migrations.